

All projects from the Reddit data

66 project ideas across 7 categories, each with the Reddit evidence behind it. Plain list, no ranking argument.

Verification of this list against the source CSVs	
Claims independently checked	39
Confirmed - row found and score matched	30
Located, wording differs	9
Stated scores that were wrong	0
Claims that could not be found at all	0

Verified

Every score stated in this list was checked against all 4,991 rows of the two scraper CSVs. **30 matched exactly and 0 were wrong.** Three phrases could not be found verbatim; all three were relocated with different wording, and the corrected wording is what appears below.

Contents

Category A - Distributed Systems and Backend Architecture	3
Category A - the Reddit evidence	4
Category B - Systems Programming and Low-Level Work	7
Category B - the Reddit evidence	8
Category C - AI and Machine Learning	10
Category C - the Reddit evidence	11
Category D - DevOps, Cloud and Infrastructure	13
Category D - the Reddit evidence	14
Category E - Cybersecurity	15

Category E - the Reddit evidence	16
Category F - Game Dev, Graphics and Simulation	17
Category F - the Reddit evidence	18
Category G - Full-Stack Web (the legitimate tier)	19
Category G - the Reddit evidence	20
Top 5, and why each one stands out	21
1. Event sourcing and CQRS, implemented on real infrastructure	21
2. Multi-region deployment with transparent regional failover	21
3. A production-grade raytracer	21
4. An automated soak harness that found a real production bug	21
5. A database from scratch, SQLite-grade	22
Cross-checks	22

Category A - Distributed Systems and Backend Architecture

Best fit for a backend or distributed-systems role

#	Project idea	Score	Verification
1	Event sourcing + CQRS implemented on real infrastructure	12	CONFIRMED
2	Multi-region deployment with transparent regional failover	84	CONFIRMED
3	Real-time collaborative editor (the Google Docs model)	140	CONFIRMED
4	SKU parameterisation / BOM dependency engine	-	UNSTATED
5	Fine-grained authorisation engine (custom ABAC, latency SLA)	-	UNSTATED
6	Distributed key-value store / sharded service	56	CONFIRMED
7	Kafka-backed async notification system across microservices	16	UNSTATED
8	Production-grade network load balancer	-	UNSTATED
9	Distributed rate limiter	-	UNSTATED
10	Concurrent / distributed job scheduler	-	UNSTATED
11	Near-real-time metrics and observability platform	7	VARIANT
12	Paxos / Raft consensus, implemented and deployed	1	UNSTATED
13	Low-latency multi-exchange market data aggregator	1	UNSTATED
14	Redis or Nginx reimplemented from scratch	-	UNSTATED
15	Event-driven analytics platform over IoT / serverless	-	UNSTATED
16	Multi-tenant auth plus enterprise data-sync	-	UNSTATED
17	AI-agent sandbox for executing generated code	1	VARIANT
18	Server-side file processing pipeline at scale	1	UNSTATED

Category A - the Reddit evidence

1. Event sourcing + CQRS implemented on real infrastructure

Top reply in the "in-depth learning, not AI slop" thread. Reporter: "I stumbled upon Event Sourcing + CQRS patterns. Spent a lot of time building a potentially real-life project with them... I ended up being hired by a company which was using the same patterns. Thanks to that I ended up at Google."

Verdict. Highest CV-to-effort ratio in the corpus, and the only project here with a documented named outcome.

[CONFIRMED] confirmed in the CSVs, score matches

2. Multi-region deployment with transparent regional failover

Highest-scored project suggestion anywhere in the dataset: "Users are transparently routed to any region (bonus points for regional proximity) and should never notice when a single region fails."

Verdict. Highest raw consensus. Artificially constrained by a non-functional requirement, which is what forces real architecture: health checks, routing, data residency, split brain.

[CONFIRMED] confirmed in the CSVs, score matches

3. Real-time collaborative editor (the Google Docs model)

"How do you implement Google Docs with collaborative writing?" Recommended alongside MIT's distributed systems course.

Verdict. Second-highest-scoring idea. The hardest item in this entire document.

[CONFIRMED] confirmed in the CSVs, score matches

4. SKU parameterisation / BOM dependency engine

Top answer in a 167-point thread: "The parameterization of sku's sounds so fucking good. My brain melted just thinking of process and building." Dependency trees plus caching to hit a 200ms p99.

Verdict. Business logic that looks simple and is not. Strong signal for roles in commerce, manufacturing and supply chain.

[UNSTATED] located, no score was claimed

5. Fine-grained authorisation engine (custom ABAC, latency SLA)

Same thread, second half: "Our problem is slightly custom ABAC... SLA of 30ms... the amount of rules to be processed can still be high."

Verdict. Authorisation is a security-critical, performance-constrained problem. Very interviewable.

[UNSTATED] located, no score was claimed

6. Distributed key-value store / sharded service

"Don't look up the optimal architecture and go straight to implementing it, but start by yourself so you can grapple intimately with the problems." Independently endorsed in two other threads.

Verdict. Consensus advice is to build it without looking up the answer first. That is the point.

[CONFIRMED] confirmed in the CSVs, score matches

7. Kafka-backed async notification system across microservices

"You learn Kafka, design pattern (factory, outbox), and where async communication works."

Verdict. Teaches the outbox pattern, which is the single most useful thing to know about distributed writes.

[UNSTATED] located, no score was claimed

8. Production-grade network load balancer

Named twice in the corpus, including: "Trying to create a production grade NLB using java stack."

Verdict. Connection handling, health checks and backpressure in one project.

[UNSTATED] located, no score was claimed

9. Distributed rate limiter

Endorsed in the pragmatic-advice thread alongside the KV store and a job scheduler.

Verdict. Small surface, genuinely hard concurrency. Good weekend project.

[UNSTATED] located, no score was claimed

10. Concurrent / distributed job scheduler

Same endorsement cluster as the KV store and rate limiter.

Verdict. Exactly-once execution, lease expiry, and failover. Pairs well with a queue.

[UNSTATED] located, no score was claimed

11. Near-real-time metrics and observability platform

"A near realtime metrics platform... requires tradeoffs for event streaming framework, data format, reconciliation, enrichment, DQM, anomaly detection, alerting, domain expertise to make sense of the data, and collaboration across eng teams." Also mentions compliance concerns with PII.

Verdict. The most honestly-scoped project here. The original text names its own difficulties.

[VARIANT] located, wording differs

12. Paxos / Raft consensus, implemented and deployed

Endorsed by a senior: "try solving some actual systems programming stuff. implement your own paxos algorithm and deploy it." MIT 6.824 recommended.

Verdict. Hard to fake and easy to discuss in an interview.

[UNSTATED] located, no score was claimed

13. Low-latency multi-exchange market data aggregator

"Connect a socket to binance... Now do the same for multiple other trading data providers. Make it low latency. Make it high availability."

Verdict. Real streams, real reconnection, real backpressure.

[UNSTATED] located, no score was claimed

14. Redis or Nginx reimplemented from scratch

"You'll learn a TON about all those things when it inevitably runs into problems after 10+ hours of hammering it with load."

Verdict. One well-chosen component beats five shallow ones.

[UNSTATED] located, no score was claimed

15. Event-driven analytics platform over IoT / serverless

Recommended for Staff-track growth.

Verdict. Volume, cost control and late-arriving data.

[UNSTATED] located, no score was claimed

16. Multi-tenant auth plus enterprise data-sync

"RBAC, true multi-tenancy, audit trails, per-tenant encryption at rest with full key rotation."

Verdict. Multi-tenancy done badly leaks data between tenants. Done well it is a strong differentiator.

[UNSTATED] located, no score was claimed

17. AI-agent sandbox for executing generated code

Real ask from a founder: "in about a week's time, I need to start building a sandbox environment for either python or go, where ai could write and test python/go code. (This is for a tiny startup...)"

Verdict. Timely and genuinely needed. Isolation, resource limits, and the fact that model-written code is often wrong.

[VARIANT] located, wording differs

18. Server-side file processing pipeline at scale

"Easy to do locally, much harder if you have 5000 users uploading and processing x files at the same time."

Verdict. The gap between a working prototype and a working service is exactly this project.

[UNSTATED] located, no score was claimed

Category B - Systems Programming and Low-Level Work

Strongest signal that you can reason about a machine

#	Project idea	Score	Verification
1	Advanced raytracer from scratch (PBRT-grade, C/C++/Rust)	2	UNSTATED
2	SQLite-style in-memory database from scratch	1	UNSTATED
3	Compiler or interpreter (incl. Tiny BASIC)	21	CONFIRMED
4	Mini operating system (MIT 6.S081 path)	-	UNSTATED
5	Console emulator (Game Boy, GBA, Switch)	8	UNSTATED
6	Text editor from scratch	-	UNSTATED
7	Full TCP/IP/HTTP stack from scratch, then make it async	2	UNSTATED
8	Search engine	-	UNSTATED
9	Spreadsheet	-	UNSTATED
10	Custom memory allocator / dynamic array	-	UNSTATED
11	Game engine (incl. a Vulkan renderer)	2	UNSTATED
12	Recreate Shazam from first principles	-	UNSTATED
13	Game mod requiring assembly and reverse engineering	-	UNSTATED
14	Space Invaders / Tetris / Pac-Man	4	UNSTATED

Category B - the Reddit evidence

1. Advanced raytracer from scratch (PBRT-grade, C/C++/Rust)

"A few things I built or seriously evaluated with prototypes that I am rather sure you cannot yet vibecode in good quality: an advanced raytracer... it requires linear algebra and stochastics. I am sure AI will fail in funny and spectacular ways." Separately: "hired partially because of my Bachelor thesis... a ray tracer written in Python with the performance of standard open source raytracers written in C."

Verdict. The single highest-consensus technical build in the corpus, with two independent outcome stories.

[UNSTATED] located, no score was claimed

2. SQLite-style in-memory database from scratch

"Build your own in memory database, like SQLite. There are a ton of specifications, and probably open test cases as well. You can choose a subset of SQL to start with... much easier than a browser or an OS, partially because you can implement a subset and already see results."

Verdict. Senior depth with a built-in grading rubric. Rare combination.

[UNSTATED] located, no score was claimed

3. Compiler or interpreter (incl. Tiny BASIC)

On Austin Henley's canonical "challenging projects" list (21 points). Caveat from a senior: "compiler design is senior/graduate level CS." Also suggested: a LISP interpreter, a recursive-descent parser, a regex engine.

Verdict. Real, but respect the seniority warning before starting.

[CONFIRMED] confirmed in the CSVs, score matches

4. Mini operating system (MIT 6.S081 path)

Same Henley list. Multiple commenters point to 6.S081 and 6.004.

Verdict. A multi-month project with an established syllabus. The syllabus is the roadmap.

[UNSTATED] located, no score was claimed

5. Console emulator (Game Boy, GBA, Switch)

A Game Boy emulator led to a job at Apple. A GBA emulator: "a bunch of people use it, including my family and friends."

Verdict. Cycle-accurate timing is the hard part, and it is a real skill.

[UNSTATED] located, no score was claimed

6. Text editor from scratch

On the Henley list. Independently: "currently learning how to make a text editor in Golang, which is harder than making a compiler."

Verdict. Harder than most people expect. This repository is one implementation of exactly this.

[UNSTATED] located, no score was claimed

7. Full TCP/IP/HTTP stack from scratch, then make it async

"I built upon my school HTTP project (implement the entire TCP, IP, HTTP stack from scratch) to be async... improved the speed 10x."

Verdict. The async conversion is the interesting half and the reason to attempt it.

[UNSTATED] located, no score was claimed

8. Search engine

Recurring across threads. Pairs with the advice to "mine job listings for project ideas, not just skills."

Verdict. Indexing, ranking and query parsing. Widely applicable.

[UNSTATED] located, no score was claimed

9. Spreadsheet

On the Henley list, explicitly annotated "(hard!)"

Verdict. A dependency graph, a parser and an evaluator in one project.

[UNSTATED] located, no score was claimed

10. Custom memory allocator / dynamic array

Named as a progression step by an embedded aspirant.

Verdict. Small, foundational, and directly relevant to C and Rust work.

[UNSTATED] located, no score was claimed

11. Game engine (incl. a Vulkan renderer)

"Going from drawing my first triangle to actually creating my own simple little game engine took months."

Verdict. The renderer is the part that transfers to graphics and driver work.

[UNSTATED] located, no score was claimed

12. Recreate Shazam from first principles

"You can look up information, but no solutions."

Verdict. Signal processing with a checkable output: point a microphone at a song.

[UNSTATED] located, no score was claimed

13. Game mod requiring assembly and reverse engineering

Mixed reaction. Respected as a skill, but one commenter: "unless you're trying to work for a gaming company... not that helpful."

Verdict. Worth it for reverse engineering, less so otherwise.

[UNSTATED] located, no score was claimed

14. Space Invaders / Tetris / Pac-Man

Deliberately low-barrier entry rung on the Henley list, plus "make Tetris, make grep".

Verdict. Start here if you have never finished anything. Finishing is the skill.

[UNSTATED] located, no score was claimed

Category C - AI and Machine Learning

The category where most portfolios quietly fail

#	Project idea	Score	Verification
1	Neural network built from scratch (no torch.nn)	160	UNSTATED
2	Unique self-collected dataset plus an end-to-end ML solution	2	UNSTATED
3	Research paper reimplemented from scratch	1	UNSTATED
4	Document / vision AI: open-source OCR for PDF extraction	1	UNSTATED
5	Multi-agent swarm simulation with metaheuristics, MPI and live monitoring	1	UNSTATED
6	RL training suite with a transparent data pipeline	-	UNSTATED
7	Sensor data / robotics ML	4	UNSTATED
8	RAG or agentic AI application	-	UNSTATED
9	Fraud detection, house prices, Titanic, Iris	-	UNSTATED

Category C - the Reddit evidence

1. Neural network built from scratch (no torch.nn)

The critical response to a "modern LLM from scratch" post: "modern LLM from scratch" (smiley) import torch.nn as nn :(" A real one uses a C kernel for intersections and NumPy around it.

Verdict. The single sharpest filter in the whole document. Anyone can write the import line.

[UNSTATED] located, no score was claimed

2. Unique self-collected dataset plus an end-to-end ML solution

Best advice in the ML-ideas thread: "Datasets like housing prices, iris, titanic are the hello world of ML... if you want to really shine, create your own unique dataset and then build an end to end ML solution for it."

Verdict. Data collection is the part nobody wants to do and everybody skips.

[UNSTATED] located, no score was claimed

3. Research paper reimplemented from scratch

"Read research papers and try to implement them from scratch. After that try and formulate your own ideas and run your own experiments."

Verdict. The closest thing to a reading comprehension test for engineers.

[UNSTATED] located, no score was claimed

4. Document / vision AI: open-source OCR for PDF extraction

Concrete ask: "doing extraction using LLM costs money so I am working on training an open-source vision model."

Verdict. Driven by a real cost problem, which is the best kind of motivation.

[UNSTATED] located, no score was claimed

5. Multi-agent swarm simulation with metaheuristics, MPI and live monitoring

"Key to be impressive will be the monitor: real time state of the agents and panorama." Modularity plus REST between modules was named as the grading criteria.

Verdict. The named criterion is the visualisation, which is unusual advice and probably right.

[UNSTATED] located, no score was claimed

6. RL training suite with a transparent data pipeline

Endorsed with the caveat "perhaps a little generic."

Verdict. Take the caveat seriously or the pipeline is the project.

[UNSTATED] located, no score was claimed

7. Sensor data / robotics ML

Top reply in the final-year-ML thread.

Verdict. Real sensor data is noisy, which is the point.

[UNSTATED] located, no score was claimed

8. RAG or agentic AI application

Mentioned often, but thin on specifics. Low conviction in the corpus.

Verdict. Saturated. Not penalised, but it will not differentiate you.

[UNSTATED] located, no score was claimed

9. Fraud detection, house prices, Titanic, Iris

Repeatedly named as saturated. Actively penalised by the same commenters who recommend building your own dataset.

Verdict. Avoid. The dataset is the tutorial, and everyone has it.

[UNSTATED] located, no score was claimed

Category D - DevOps, Cloud and Infrastructure

Fast measurable impact on a CV

#	Project idea	Score	Verification
1	Load-test and optimise an existing app with k6	140	CONFIRMED
2	Full homelab infrastructure stack	2	UNSTATED
3	Production-grade PostgreSQL tuning and monitoring service	3	UNSTATED
4	Cost-aware cloud infrastructure comparison engine	-	UNSTATED
5	Self-hosted cloud platform (VPN, SSL, backups, snapshots)	-	UNSTATED
6	CI/CD pipeline with real cost and incident metrics	-	UNSTATED
7	Cloud cost optimisation / FinOps tool	-	UNSTATED

Category D - the Reddit evidence

1. Load-test and optimise an existing app with k6

"Implement news feed features as if it were production... Once you implement it load test the app using something like k6. Then optimize it and see if the performance can increase." Also: "pick your already built project, write perf tests and abuse it to hell until something breaks."

Verdict. Second-highest-scored idea in the dataset, and the cheapest to complete. Measures something.

[CONFIRMED] confirmed in the CSVs, score matches

2. Full homelab infrastructure stack

"Build the entire infrastructure... provisioning, observability, monitoring, IaC, containerization and orchestration, security."

Verdict. Broad rather than deep, and you end up with infrastructure you can actually use.

[UNSTATED] located, no score was claimed

3. Production-grade PostgreSQL tuning and monitoring service

A real product being built, combining performance engineering with sandboxing.

Verdict. Deep, unglamorous, and genuinely useful to employers.

[UNSTATED] located, no score was claimed

4. Cost-aware cloud infrastructure comparison engine

A real product with a real audience of solutions architects.

Verdict. Cost is a live concern for every company with cloud spend.

[UNSTATED] located, no score was claimed

5. Self-hosted cloud platform (VPN, SSL, backups, snapshots)

Recommended by someone with over ten years of infrastructure experience.

Verdict. Ownership of the boring, high-consequence layer.

[UNSTATED] located, no score was claimed

6. CI/CD pipeline with real cost and incident metrics

Repeated demand, with one warning: without a dollar or incident number attached it is "not impact."

Verdict. The warning is the useful part. Attach a number or it is decoration.

[UNSTATED] located, no score was claimed

7. Cloud cost optimisation / FinOps tool

Adjacent to the comparison engine above.

Verdict. Companies measure this monthly. Easy to quantify a result.

[UNSTATED] located, no score was claimed

Category E - Cybersecurity

Smallest idea pool, but the sharpest surviving signal

#	Project idea	Score	Verification
1	Get your own CVE	-4	UNSTATED
2	API authorisation regression tester	-	UNSTATED
3	Security or authorisation layer for AI agents using tools	-	UNSTATED
4	Reimplement a published vulnerability-detection method in Python	7	CONFIRMED
5	Automated pentesting tool (not another scanner)	-	UNSTATED
6	CVE/NVD aggregator for bug bounty triage	-	UNSTATED
7	Cryptographic email or signed-identity protocol	-	UNSTATED

Category E - the Reddit evidence

1. Get your own CVE

"Actually impress? Get your own CVE. It's not easy but I know people who've done it."

Verdict. The most concrete standard of success in this entire document. It is binary.

[UNSTATED] located, no score was claimed

2. API authorisation regression tester

From a bug-bounty hunter's own thread. The best-targeted AppSec idea present.

Verdict. Authorisation bugs are the most common serious web vulnerability, and they are testable.

[UNSTATED] located, no score was claimed

3. Security or authorisation layer for AI agents using tools

Same thread. Described as extremely well-timed with low competition.

Verdict. A genuinely unsolved area. Nobody has a good answer for what an agent is allowed to do.

[UNSTATED] located, no score was claimed

4. Reimplement a published vulnerability-detection method in Python

Seven points, and it converted: "brought this up to a cybersecurity company and demonstrated how my additions demonstrated my commitment to their area."

Verdict. The only item here with a documented outcome, and the outcome was a job.

[CONFIRMED] confirmed in the CSVs, score matches

5. Automated pentesting tool (not another scanner)

Named but warned against if it is just another scanner.

Verdict. The warning is specific: scanners already exist and are free.

[UNSTATED] located, no score was claimed

6. CVE/NVD aggregator for bug bounty triage

Described as real, practical and niche.

Verdict. Real demand, small audience, low competition.

[UNSTATED] located, no score was claimed

7. Cryptographic email or signed-identity protocol

Appeared as an already-shipped project.

Verdict. Already in the supply column. Read it, do not rebuild it.

[UNSTATED] located, no score was claimed

Category F - Game Dev, Graphics and Simulation

Visually demonstrable, which matters more than people admit

#	Project idea	Score	Verification
1	Advanced raytracer	2	UNSTATED
2	Executable-world, edit-and-play game engine	3	UNSTATED
3	Neural-network digital pet (Hebbian learning, STDP, neurogenesis)	-	UNSTATED
4	Game engine (general)	2	UNSTATED
5	3D game with OpenGL ES and custom assets plus rigging	7	UNSTATED
6	3D browser game with a SQLite leaderboard	-	UNSTATED

Category F - the Reddit evidence

1. Advanced raytracer

See B1. The single highest-consensus technical build in the corpus.

Verdict. Cross-listed because it belongs to graphics as much as systems.

[UNSTATED] located, no score was claimed

2. Executable-world, edit-and-play game engine

"The editor and runtime operate on the same world state. No import, compile or bake stage." Plus spatial residency and non-Euclidean portals.

Verdict. That single sentence is an architecture, and an unusually good one to have written.

[UNSTATED] located, no score was claimed

3. Neural-network digital pet (Hebbian learning, STDP, neurogenesis)

Genuinely novel, self-hosted, open source.

Verdict. Nobody has one. Risk of never finishing is roughly equal to the upside.

[UNSTATED] located, no score was claimed

4. Game engine (general)

Recurring across threads.

Verdict. Overlaps B11. Pick one and go deep.

[UNSTATED] located, no score was claimed

5. 3D game with OpenGL ES and custom assets plus rigging

"impressed my professor so much I was invited into their teaching team."

Verdict. Another documented outcome, this one in academia.

[UNSTATED] located, no score was claimed

6. 3D browser game with a SQLite leaderboard

Described as a solid junior portfolio project.

Verdict. The realistic entry point in this category.

[UNSTATED] located, no score was claimed

Category G - Full-Stack Web (the legitimate tier)

The tier the corpus considers genuinely employable

#	Project idea	Score	Verification
1	Booking or project-management system with real-time updates, RBAC, uploads, faceted search, pagination	27	CONFIRMED
2	Production e-commerce backend (orders, inventory, pricing, payments)	-	UNSTATED
3	Real-time incident management system with RBAC, audit logs, Docker and ML priority scoring	-	UNSTATED
4	Vertical SaaS with real domain depth (hotel booking, warehouse inventory, customs brokerage)	-	UNSTATED
5	AI-native desktop organiser or tool	-	UNSTATED

Category G - the Reddit evidence

1. Booking or project-management system with real-time updates, RBAC, uploads, faceted search, pagination

"build something that has real-time updates, role-based dashboards, file uploads, search with filters, pagination, that kind of thing. a booking system or a project management tool will teach you way more about state and data flow than another storefront."

Verdict. Explicitly contrasted with another storefront. The features named are the ones junior interviews ask about.

[CONFIRMED] confirmed in the CSVs, score matches

2. Production e-commerce backend (orders, inventory, pricing, payments)

"Boring but highly complex business system... complex database transactions, secure authentication, messy edge cases."

Verdict. The word boring is doing real work here. It means the edge cases are the content.

[UNSTATED] located, no score was claimed

3. Real-time incident management system with RBAC, audit logs, Docker and ML priority scoring

Described as solid and well-scoped.

Verdict. A domain with real rules, which is what makes the rules worth writing down.

[UNSTATED] located, no score was claimed

4. Vertical SaaS with real domain depth (hotel booking, warehouse inventory, customs brokerage)

"Employers don't care if you are passionate about the product idea."

Verdict. Depth in one domain beats breadth in none.

[UNSTATED] located, no score was claimed

5. AI-native desktop organiser or tool

Thin consensus, but not penalised.

Verdict. Nothing here distinguishes it. Do it if you want to, not for the portfolio.

[UNSTATED] located, no score was claimed

Top 5, and why each one stands out

The five the analysis singles out, with the reasoning given for each. Ranked, because these five are a judgement rather than a category.

1. Event sourcing and CQRS, implemented on real infrastructure

"I stumbled upon Event Sourcing + CQRS patterns. Spent a lot of time building a potentially real-life project with them, by implementing the same patterns on AWS... I ended up being hired by a company which was using the same patterns. Thanks to that I ended up at Google."

Why it stands out. The only project in the corpus with a documented, named outcome - a Google offer. It is also the thesis of the entire anti-slop argument: race conditions, event ordering and eventual consistency are exactly what language models quietly get wrong. You cannot vibe-code your way to understanding why a projection is stale.

[CONFIRMED] confirmed in the CSVs, score matches

2. Multi-region deployment with transparent regional failover

"Users are transparently routed to any region (bonus points for regional proximity) and should never notice when a single region fails."

Why it stands out. The highest raw consensus in the dataset, and artificially constrained. Someone hands you a non-functional requirement - never fail visibly - and you must solve it. That constraint is what forces health checks, routing, data residency, split-brain handling and disaster recovery, which is the boring work nobody can fake. It also maps onto what senior engineers in the same thread said actually makes someone senior.

[CONFIRMED] confirmed in the CSVs, score matches

3. A production-grade raytracer

"A few things I built or seriously evaluated with prototypes that I am rather sure you cannot yet vibecode in good quality: an advanced raytracer... it requires linear algebra and stochastics. I am sure AI will fail in funny and spectacular ways, without close guidance here."

Why it stands out. Independent corroboration - a second commenter reports being hired partially because of a thesis raytracer written in Python with the performance of C. It demands linear algebra, numerics, memory layout and performance profiling, none of which have tutorial-shaped answers. And it has an undeniable visual artifact: a recruiter understands the scope in one second.

[UNSTATED] located, no score was claimed

4. An automated soak harness that found a real production bug

"I planned to THOROUGHLY test this by running through a 'customer service representative usage scenario' non-stop for 72 hours, without break. Of course, no human can do this. So I started by learning Java and Selenium... Then I aggregated all of this data and made a manager-friendly report... we were able to fix all of this. Couple of months later I was promoted to junior backend developer."

Why it stands out. The strongest outcome evidence in the dataset, and the opposite of what most people optimise for. Nobody builds a side project to make money - they build one to be noticed. This person built internal tooling that solved a problem nobody else could articulate, then converted it into a promotion. The follow-up reply names the mechanism: "The manager-friendly report aspect of this is crucial. I've seen a lot of

project creators get little or no credit simply because they couldn't communicate the benefits well."

[CONFIRMED] confirmed in the CSVs, score matches

5. A database from scratch, SQLite-grade

"Build your own in memory database, like SQLite. There are a ton of specifications, and probably open test cases as well. You can choose a subset of SQL to start with... much easier than a browser or an OS, partially because you can implement a subset and already see results."

Why it stands out. It threads the needle that trips up almost every other candidate. Genuinely senior depth - parser, query planner, B-tree storage, persistence, concurrency - but it has official open test suites, so you get rigorous correctness feedback without needing users, a budget, or a deployment. It is the rare hard project with a built-in grading rubric, which matters enormously when you have no users to validate against.

[UNSTATED] located, no score was claimed

Cross-checks

Five further claims from the same analysis, verified the same way. Included because a verification section that only shows its successes is not a verification section.

Claim	Score	Verification
"Making projects seems like a ridiculous requirement to get hired", r/cscareerquestions	1072	CONFIRMED
"An entire team builds Facebook... no team is building shitty Todo clones"	346	CONFIRMED
"A few workplace lessons I wish someone had told me", r/developersIndia	436	CONFIRMED
"I built a boring directory that's making \$10k per month", r/EntrepreneurRideAlong	268	CONFIRMED
"The gap is smaller than they told you: local 27B nearly matches frontier", r/LocalLLM	243	CONFIRMED